Chapter 12

Using Redux to Manage State

Now that we are using a reducer to manage state in our React app, we've come up against an issue with integrating our asynchronous actions with our reducer. We've chosen to keep our asynchronous actions outside our reducer, which means we need to remember to trigger the asynchronous actions when we call the reducer.

This brings us to a tool called Redux[72] and its associated libraries, React Toolkit[73] and React Redux.[74] Redux is a more complex version of the reducer and store pattern we've been working with. I don't cover a lot of the extended complexity here, but Redux has support for more complex combinations of reducers and actions than we need, at least at the moment. It bills itself as a "Predictable State Container for JS Apps," which for our purposes is the marketing-speak version of what we've been talking about in the previous chapter. By "state container," it's referring to what we've been calling a *store*, and by "predictable," it's referring to the way reducers give you state objects that won't change behind your back.

React Redux is the library that binds it to React, and Redux Toolkit bills itself as "The official, opinionated, batteries-included toolset for efficient Redux development," which basically means that they took developer feedback of how Redux was actually used and provided a more useful API and made it the recommended way to use Redux. We'll be using the toolkit methods here.

There are many storage and reducer options in the React ecosystem these days. I'm sticking to Redux here, but there may be other options that fit your project.